

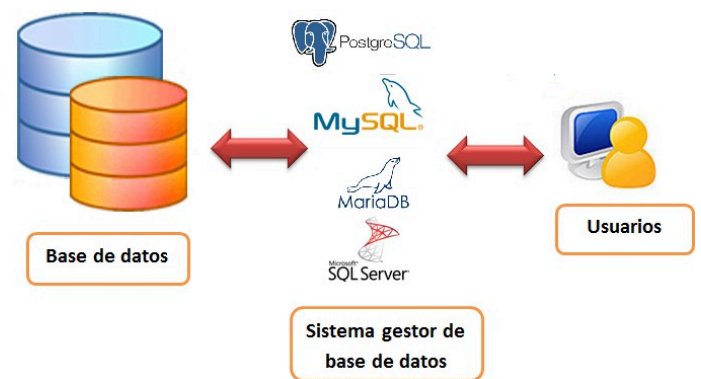
Base de Datos

Guía de Trabajos Prácticos N° 1

Para realizar esta guía será necesaria la instalación de un sistema de base de datos relacional (**RDBMS**) y un Sistema de Gestión de Base de Datos Relacionales en español (**SGBDR**). Este último es un software que permite crear, actualizar y administrar bases de datos relacionales.

Para nuestra materia vamos a usar **MariaDB**, que es completamente compatible con **MySQL**. A continuación, presentamos el contexto histórico de por qué existen estas dos versiones de una misma base de datos y cuál consideramos que deberíamos usar con estudiantes de Ingeniería

Informática. Sin embargo, son libres de usar cualquiera de las dos versiones, la sintaxis trabajada en la práctica será compatible con ambos motores de bases de datos:



La Historia de MySQL y MariaDB

★ El Nacimiento de MySQL (1995-2008)

MySQL nació en 1995 de la mano de la empresa sueca MySQL AB. Fue diseñado para ser un **sistema de gestión de bases de datos relacionales** (RDBMS) rápido, robusto y fácil de usar. Su éxito se basó en su naturaleza de **código abierto**, lo que lo hizo muy popular entre los desarrolladores web, y se convirtió en la base de muchas aplicaciones importantes. La combinación de **Linux, Apache, MySQL y PHP/Perl/Python** (conocido como la pila **LAMP**) fue clave para el desarrollo de la web moderna.

En 2008, la empresa fue adquirida por Sun Microsystems. Esto

generó cierta incertidumbre en la comunidad de código abierto, pero la situación se intensificó poco después.

★ La Adquisición por Oracle y el Nacimiento de MariaDB (2009 en adelante)

En 2009, **Oracle Corporation** anunció su intención de comprar Sun Microsystems. La noticia preocupó a la comunidad, ya que Oracle es una de las mayores empresas de bases de datos propietarias del mundo. Muchos temieron que Oracle pusiera fin al desarrollo de MySQL como un proyecto de código abierto o que restringiera su uso.

Para asegurar la continuidad del proyecto y su filosofía de libre uso, **Michael "Monty" Widenius**, uno de los cofundadores de MySQL, creó un *fork* (una bifurcación) de la base de código. Así nació **MariaDB**, llamado así en honor a su hija menor, Maria (MySQL se llamaba así por su otra hija, My).

La comunidad de código abierto se unió a MariaDB, y muchos desarrolladores clave de MySQL se unieron al proyecto. Esto aseguró que MariaDB continuara siendo un reemplazo directo y compatible con MySQL, pero bajo una **licencia de software libre y gobernanza comunitaria**.

MySQL vs. MariaDB: Recomendaciones para Estudiantes

La elección entre MySQL y MariaDB es un tema relevante, especialmente desde la perspectiva de licenciamiento.

★ Desde un punto de vista educativo

Para el aprendizaje, **la elección entre uno y otro es casi irrelevante**. La sintaxis SQL, la administración básica y la lógica de las bases de datos son **prácticamente idénticas**. Un estudiante que aprende a usar MySQL no tendrá problemas para trabajar con MariaDB, y viceversa. La mayoría de los cursos,

tutoriales y libros de texto usan los mismos ejemplos y comandos para ambos.

★ Desde un punto de vista de licenciamiento y uso libre

Aquí es donde radica la principal diferencia y la razón por la que **MariaDB** suele ser la opción preferida para proyectos de código abierto y educativos.

1. **MySQL**: Actualmente, pertenece a Oracle. Aunque sigue ofreciendo una versión de código abierto (la **MySQL Community Edition**), también tiene una versión **Enterprise** con funcionalidades propietarias. Su licencia ha generado preocupaciones sobre el futuro del proyecto y la posibilidad de cambios en las condiciones de uso.
2. **MariaDB**: Pertenece y es gestionado por la **MariaDB Foundation**. Esto asegura que el proyecto mantenga su naturaleza de **software libre** bajo la licencia GPL (GNU General Public License). La gobernanza comunitaria garantiza que las decisiones sobre el futuro del proyecto se tomen en el mejor interés de la comunidad, sin el riesgo de ser controladas por una sola corporación.

★ Conclusión y Recomendación

Para un estudiante de ingeniería informática, **la recomendación es usar MariaDB**.

No solo porque la experiencia de aprendizaje es la misma que con MySQL, sino porque el uso de MariaDB promueve los valores de **software libre y la independencia tecnológica**, que son pilares fundamentales de la ingeniería de software moderna. Les enseña la importancia de la comunidad, la transparencia en el desarrollo y la sostenibilidad de los proyectos de código abierto.

INSTALACIÓN DE MARIADB EN GITHUB CODESPACES

1. Crear el Codespace

1. Ve a tu repositorio en GitHub (o crea uno nuevo).
2. Haz clic en el botón verde **<> Code**.
3. Selecciona la pestaña **Codespaces** y haz clic en **Create codespace on main**.
4. Espera a que cargue el entorno en el navegador.

2. Instalar MariaDB

Una vez que abra la terminal en la parte inferior, ejecuta los siguientes comandos para actualizar los paquetes e instalar el servidor:

Terminal de Bash

```
sudo apt update  
sudo apt install mariadb-server -y
```

3. Iniciar el Servicio

En Codespaces, los servicios no siempre arrancan solos. Debes iniciarlo manualmente:

Terminal de Bash

```
sudo service mariadb start
```

4. Configuración de Seguridad (Opcional pero recomendado)

Si quieres definir una contraseña y limpiar usuarios de prueba, ejecuta:

Terminal de Bash

```
sudo mysql_secure_installation
```

Tip: Para un entorno de desarrollo rápido, puedes saltarte esto y entrar directamente como root (sudo).

5. Acceder a MariaDB desde la Terminal

Para entrar a la consola de MariaDB, usa:

Terminal de Bash

```
sudo mariadb
```

Comandos básicos de supervivencia:

Una vez dentro del prompt de MariaDB (**MariaDB [(none)]>**), puedes probar lo siguiente:

- Ver bases de datos: **SHOW DATABASES;**
 - Crear una nueva: **CREATE DATABASE mi_proyecto;**
 - Usar la base de datos: **USE mi_proyecto;**
 - Salir: **exit;**
-

Modalidad Tutoría

Para realizar esta guía será necesaria la instalación de un servidor de base de datos relacional (RDBMS). Un SGBDR (en español, Sistema de Gestión de Base de Datos Relacionales) es un software que permite crear, actualizar y administrar bases de datos relacionales (ver Guía de Trabajos Prácticos N°1). La primera parte de esta guía repite la creación de usuario y base de datos de la Guía 1.

Parte 1 Práctica Asistida

1. Ejemplo didáctico: Base de datos para una tienda de libros

a. Analice el siguiente modelo de la Base de Datos

(Listado y detalle de tablas)

→ Customers (Clientes)

- ◆ **id** (int, PK, auto_increment)
- ◆ **name** (varchar)
- ◆ **email** (varchar)
- ◆ **phone_number** (varchar)
- ◆ **address** (text)

→ Books (Libros)

- ◆ **id** (int, PK, auto_increment)
- ◆ **title** (varchar)
- ◆ **author** (varchar)
- ◆ **price** (decimal)
- ◆ **stock_quantity** (int)

→ Orders (Pedidos)

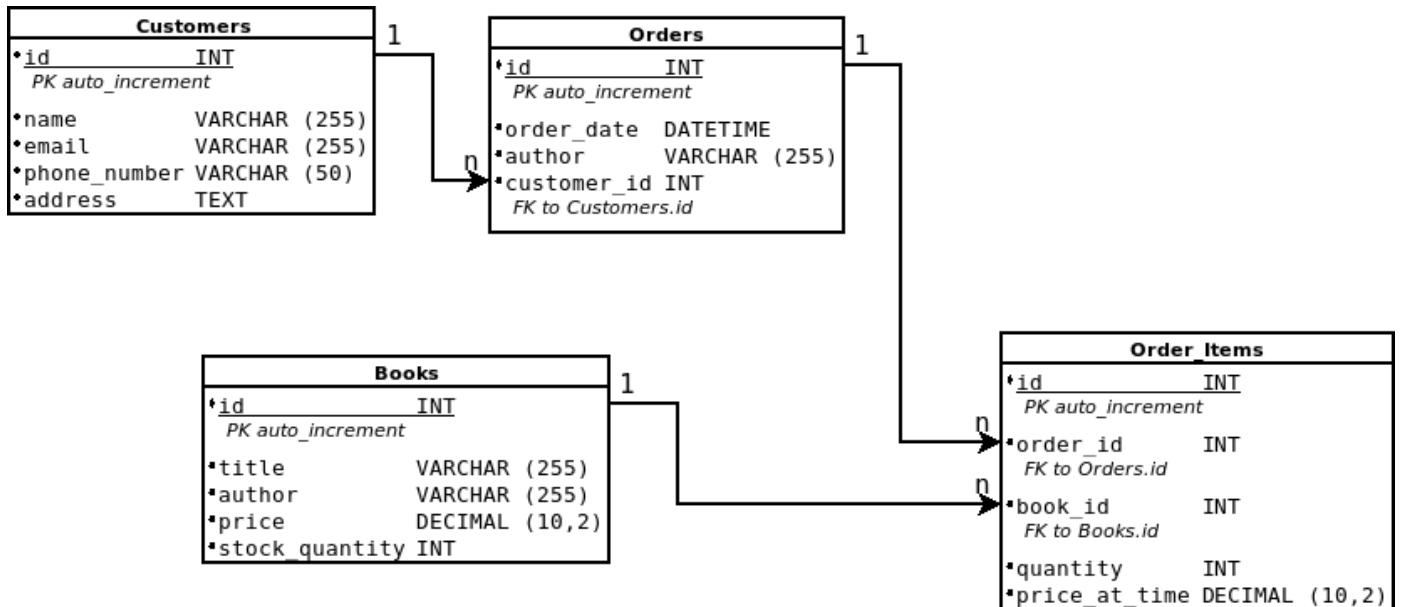
- ◆ **id** (int, PK, auto_increment)
- ◆ **order_date** (datetime)
- ◆ **customer_id** (int, FK hacia Customers.id)

→ Order_Items (Detalles/Ítems de los pedidos)

- ◆ **id** (int, PK, auto_increment)
- ◆ **order_id** (int, FK hacia Orders.id)
- ◆ **book_id** (int, FK hacia Books.id)
- ◆ **quantity** (int)
- ◆ **price_at_time** (decimal)

b. Compare con el Diagrama de Base de Datos:

Identifique los elementos presentes en el diagrama y en la descripción de la base de datos bookstoredb. Observe las convenciones de uso de mayúscula y minúscula en los nombres de tablas, tipos y campos. Analice los valores de almacenamiento asociados a cada tipo. Tome apunte de lo identificado.



2. Lenguaje de Definición de Datos (DDL) Creación de la base de datos, credenciales y creación de tablas (Script SQL). Ejecute las siguientes sentencias y revise los resultados en Adminer:

a. Volver a estado cero si existiesen usuarios y base de datos creadas previamente con el mismo nombre

```

REVOKE ALL PRIVILEGES, GRANT OPTION FROM 'bookstore_user'@'localhost';
DROP USER IF EXISTS 'bookstore_user'@'localhost';
DROP DATABASE IF EXISTS bookstoredb;
FLUSH PRIVILEGES;
  
```

```

Windows PowerShell
MariaDB [(none)]> show databases;
+-----+
| Database |
+-----+
| bookstoredb |
| information_schema |
| mysql |
| performance_schema |
| sys |
+-----+
5 rows in set (0.027 sec)

MariaDB [(none)]> REVOKE ALL PRIVILEGES, GRANT OPTION FROM 'bookstore_user'@'localhost';
Query OK, 0 rows affected (0.011 sec)

MariaDB [(none)]> DROP USER IF EXISTS 'bookstore_user'@'localhost';
Query OK, 0 rows affected (0.010 sec)

MariaDB [(none)]> DROP DATABASE IF EXISTS bookstoredb;
Query OK, 0 rows affected (0.018 sec)

MariaDB [(none)]> FLUSH PRIVILEGES;
Query OK, 0 rows affected (0.002 sec)

MariaDB [(none)]> show databases;
+-----+
| Database |
+-----+
| information_schema |
| mysql |
| performance_schema |
| sys |
+-----+
4 rows in set (0.001 sec)
  
```

Verificamos la eliminación

b. Crear la Base de Datos (EJECUTAR COMO ROOT)

Crea una base de datos llamada **bookstoredb** con el cotejamiento **utf8_unicode_ci**.

```
CREATE DATABASE bookstoreb CHARACTER SET utf8 COLLATE utf8_unicode_ci;
```

b. Crear un Usuario en MySQL (EJECUTAR COMO ROOT)

Ahora creamos un usuario llamado **bookstore_user** con la contraseña **12345** (a los fines prácticos usamos una contraseña insegura y fácil de recordar):

```
CREATE USER 'bookstore_user'@'localhost' IDENTIFIED BY '12345';
```

c. Asignar Permisos al Usuario (EJECUTAR COMO ROOT)

Otorgamos todos los permisos sobre la base de datos **bookstoredb** al usuario **bookstore_user**:

```
GRANT ALL PRIVILEGES ON bookstoreb.* TO 'bookstore_user'@'localhost';
```

Para que los cambios surtan efecto, ejecutamos:

```
FLUSH PRIVILEGES;
```

d. Verificar los Permisos Asignados (EJECUTAR COMO ROOT)

Podemos verificar qué permisos tiene **bookstore_user** con:

```
SHOW GRANTS FOR 'bookstore_user'@'localhost';
```

Esto mostrará algo como:

```
GRANT ALL PRIVILEGES ON `bookstoredb`.* TO 'bookstore_user'@'localhost'
```

d. Salir del Usuario Root

Ahora salimos de la sesión actual en MySQL (si hubiéramos ejecutado esto en una terminal, sino no es necesario):

```
EXIT;
```

e. Iniciar Sesión con el Nuevo Usuario

Ahora iniciamos sesión en MySQL con **bookstore_user** (solo en una terminal, en adminer este proceso es en entorno gráfico):

```
mysql -u bookstore_user -p
```

Cuando pida la contraseña, ingresamos:

```
12345
```

A partir de aquí crear las tablas:

```
-- 5 Usar la base de datos bookstore (esto solo es necesario  
-- cuando usamos la terminal)
```

```
USE bookstoredb;
```

```
-- Crear la tabla Customers
```

```
CREATE TABLE Customers (  
    id INT AUTO_INCREMENT PRIMARY KEY,  
    name VARCHAR(255),  
    email VARCHAR(255) UNIQUE,  
    phone_number VARCHAR(50),  
    address TEXT  
);
```

```
-- Crear la tabla Books
```

```
CREATE TABLE Books (  
    id INT AUTO_INCREMENT PRIMARY KEY,  
    title VARCHAR(255),  
    author VARCHAR(255),  
    price DECIMAL(10, 2),  
    stock_quantity INT  
);
```

```
-- Crear la tabla Orders
```

```
CREATE TABLE Orders (  
    id INT AUTO_INCREMENT PRIMARY KEY,  
    order_date DATETIME,  
    customer_id INT,  
    FOREIGN KEY (customer_id) REFERENCES Customers(id)  
);
```

```
-- Crear la tabla Order_Items
```

```
CREATE TABLE Order_Items (  
    id INT AUTO_INCREMENT PRIMARY KEY,  
    order_id INT,  
    book_id INT,  
    quantity INT,  
    price_at_time DECIMAL(10, 2),  
    FOREIGN KEY (order_id) REFERENCES Orders(id),  
    FOREIGN KEY (book_id) REFERENCES Books(id)  
);
```

Ejecutar estas sentencias como usuario bookstore_user: Insertar y actualizar Datos (Lenguaje de Manipulación de datos DML).

-- Insertar clientes

```
INSERT INTO Customers (name, email, phone_number, address) VALUES
('John Doe', 'john.doe@example.com', '555-1234', '123 Main St'),
('Jane Smith', 'jane.smith@example.com', '555-5678', '456 Oak Ave');
```

-- Insertar libros

```
INSERT INTO Books (title, author, price, stock_quantity) VALUES
('The Great Gatsby', 'F. Scott Fitzgerald', 10.99, 100),
('1984', 'George Orwell', 8.99, 50);
```

-- Insertar pedidos

```
INSERT INTO Orders (order_date, customer_id) VALUES
(NOW(), 1),
(NOW(), 2);
```

-- Insertar detalles de los pedidos

```
INSERT INTO Order_Items (order_id, book_id, quantity, price_at_time) VALUES
(1, 1, 2, 10.99),
(2, 2, 1, 8.99);
```

3. Lenguaje de Consulta de Datos (DQL): Ejemplos de consultas básicas para recuperar información de las tablas, ejecute las siguientes sentencias y revise los resultados en Adminer, haga la misma prueba en la terminal:

a. Obtener todos los clientes:

- `SELECT * FROM Customers;`

```
MariaDB [bookstoredb]> SELECT * FROM Customers;
+----+-----+-----+-----+-----+
| id | name      | email                      | phone_number | address      |
+----+-----+-----+-----+-----+
| 1  | John Doe  | john.doe@example.com      | 555-1234     | 123 Main St |
| 2  | Jane Smith | jane.smith@example.com    | 555-5678     | 456 Oak Ave |
+----+-----+-----+-----+-----+
```

b. Obtener todos los pedidos realizados por un cliente específico:

- `SELECT * FROM Orders WHERE customer_id = 1;`

c. Obtener todos los libros que están en un pedido específico:

- `SELECT b.title, oi.quantity, oi.price_at_time
FROM Order_Items AS oi
JOIN Books AS b ON oi.book_id = b.id
WHERE oi.order_id = 1;`

d. Obtener el total de un pedido (usando la cantidad de libros y el precio en el momento de la compra):

- `SELECT SUM(oi.quantity * oi.price_at_time) AS total
FROM Order_Items AS oi
WHERE oi.order_id = 1;`

4. Otros ejemplos DML. Nuevamente ejecute las siguientes sentencias y revise los resultados en Adminer, haga la misma prueba en la terminal, (probablemente deba ajustar alguna condición para probar las mismas sentencias, lea el apartado DATO IMPORTANTE):

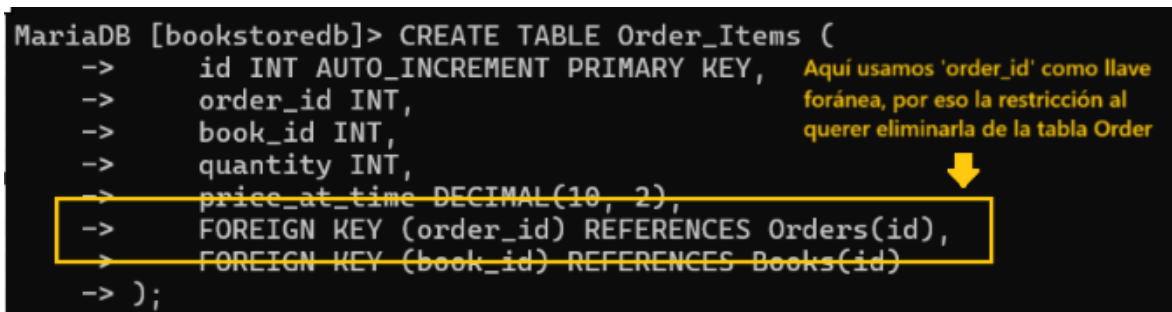
a. Actualizar el stock de un libro:

- `UPDATE Books SET stock_quantity = stock_quantity - 1 WHERE id = 1;`

b. Eliminar un pedido:

- `DELETE FROM Orders WHERE id = 2;`

DATO IMPORTANTE: Al eliminar van a obtener un error-> `Cannot delete or update a parent row: a foreign key constraint fails (`bookstoreDB`.`Order_Items`, CONSTRAINT `Order_Items_ibfk_1` FOREIGN KEY (`order_id`) REFERENCES `Orders` (`id`))`



```
MariaDB [bookstoredb]> CREATE TABLE Order_Items (
->   id INT AUTO_INCREMENT PRIMARY KEY,
->   order_id INT,
->   book_id INT,
->   quantity INT,
->   price_at_time DECIMAL(10, 2),
->   FOREIGN KEY (order_id) REFERENCES Orders(id),
->   FOREIGN KEY (book_id) REFERENCES Books(id)
-> );
```

Eso es correcto, ya que es la función de la restricción de la clave foránea. Hay formas de que se permita el borrado y la actualización cuando existe esta restricción. Esto se realiza utilizando las restricciones combinadas con las opciones para actualizar y eliminar en cascada:

ON DELETE CASCADE ON UPDATE CASCADE

Entonces para poder eliminar en cascada modificamos la tablas correspondientes del siguiente modo:

```
ALTER TABLE `Orders`
```

```
DROP FOREIGN KEY `Orders_ibfk_1`,
```

```
ADD FOREIGN KEY (`customer_id`) REFERENCES `Customers` (`id`) ON DELETE CASCADE ON UPDATE CASCADE;
```

```
ALTER TABLE `Order_Items`
```

```
DROP FOREIGN KEY `Order_Items_ibfk_1`,
```

```
ADD FOREIGN KEY (`order_id`) REFERENCES `Orders` (`id`) ON DELETE CASCADE ON UPDATE CASCADE;
```

Nota: `ibfk` = *InnoDB Foreign Key*. Es una convención de nomenclatura.